COE 292 – Final Project Report

Course: COE 292 – Introduction to Artificial Intelligence
Team ID: S12_6
Team Members: [SAAD ALDOSSARY, HASSAN ALFAIFI, JAWAD ALHARBI]
Dataset Selected: Car Evaluation Dataset; Source: UCI Machine Learning Repository; Purpose: to assess car acceptability.

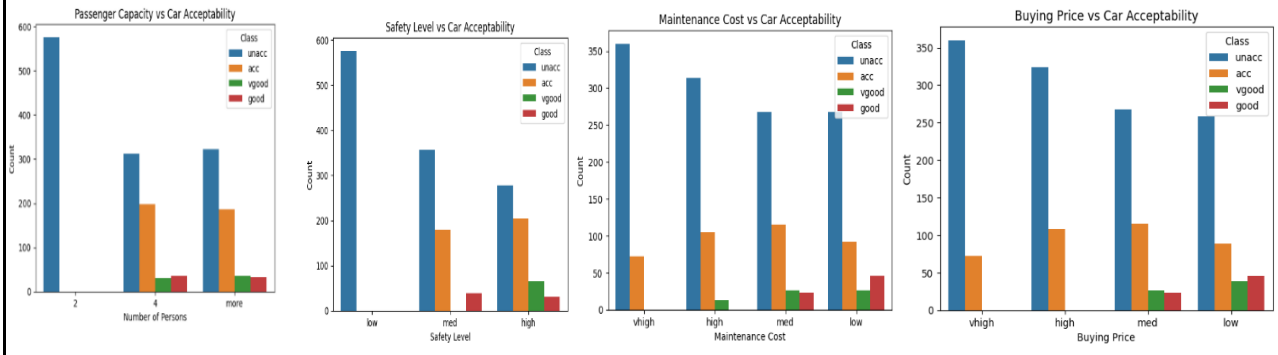
1. Dataset Exploration and Preparation (10 points)

• Number of samples: **1728 instances**
• Number of features: **6 categorical features** (buying price, maintenance cost, number of doors, passenger capacity, luggage boot size, safety level)
• Number of classes**: 4 classes** (unacceptable (unacc), acceptable (acc), good, very good (vgood))
• Class distribution (samples per class): unacc: 1210 samples, acc: 384 samples, good: 69 samples, vgood: 65 samples.

• **Preprocessing and scaling steps applied:** Categorical features were encoded using **One-Hot Encoding**, as it avoids introducing artificial ordinal relationships between categories. This choice was supported by experimental results showing improved performance for KNN, SVM, and MLP. The dataset was divided into features (X) and target (y). Since these models are sensitive to feature scale, **Standard Scaling** was applied after encoding. To prevent data leakage, both encoding and scaling were performed within each fold by fitting them on the training data and applying them to both the training and testing sets.

• **5-fold (stratified) cross-validation setup**: A 5-fold stratified cross-validation was used to evaluate the models. The dataset was divided into five folds while preserving the class distribution in each fold. In each iteration, four folds were used for training and one fold for testing. This process was repeated five times, and the results were obtained by averaging the performance across all folds.

Data Visualization (Required):



Note: Scatter plots were used after encoding the categorical features to satisfy the requirement. However, due to the categorical nature of the dataset and limited distinct values, they resulted in overlapping points and reduced interpretability. Therefore, count plots and feature-to-class visualizations were used to provide clearer and more meaningful insights.

2. Classification Algorithm 1 – K-Nearest Neighbors (10 points)

• KNN uses Euclidean distance, and Standard Scaling was applied after One-Hot Encoding to normalize feature values.

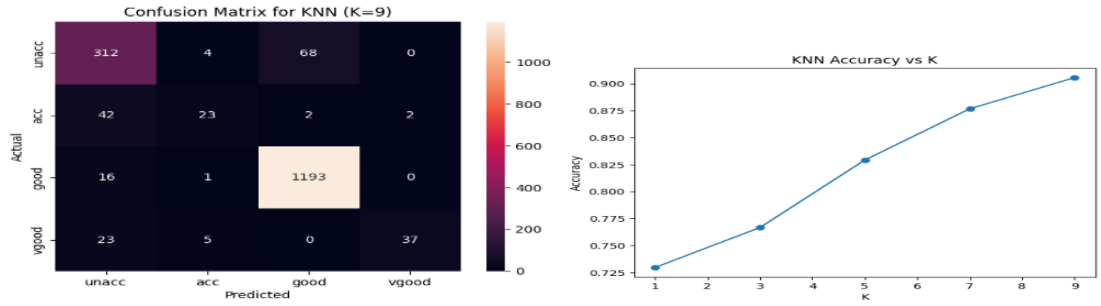
• The optimal value of K was 9 because it yielded the highest accuracy and had more stable results than smaller K values. The tested values of K were (1, 3, 5, 7, 9), as shown in the table below.

5-fold cross-validation results (mean ± std):

K	Accuracy	F1 Score	Precision	Sensitivity(Recall)	Specificity
1	0.7298 ± 0.0413	0.7295 ± 0.0397	0.7314 ± 0.0397	0.4490 ± 0.0791	0.8618 ± 0.0216
3	0.7667 ± 0.0409	0.7533 ± 0.0455	0.7508 ± 0.0590	0.4409 ± 0.0787	0.8724 ± 0.0304
5	0.8293 ± 0.0206	0.8080 ± 0.0253	0.7982 ± 0.0315	0.4697 ± 0.0442	0.8939 ± 0.0149
7	0.8767 ± 0.0241	0.8677 ± 0.0260	0.8736 ± 0.0243	0.6087 ± 0.0689	0.9321 ± 0.0149
9	**0.9057 ± 0.0102**	**0.8981 ± 0.0121**	**0.9014 ± 0.0115**	**0.6751 ± 0.0553**	**0.9493 ± 0.0056**

Performance Evaluation:

- *The model was evaluated using Accuracy, Precision, Recall (Sensitivity), F1-score, and Specificity, along with the confusion matrix.*
- *5-fold cross-validation was used to improve reliability and reduce overfitting.*
- *At **K = 9**, the model achieved high accuracy and strong precision and F1-score, indicating good overall performance.*
- *The sensitivity (0.6751) shows that the model is less effective in classifying minority classes.*
- *The specificity (0.9493) indicates that the model performs very well in identifying negative cases.*
- *This suggests KNN is biased toward majority classes due to class imbalance.*



➢ **The confusion matrix shows that most predictions are correct, with misclassifications mainly occurring in smaller classes such as "acc" and "vgood."**

3. Classification Algorithm 2 – Support Vector Machine (10 points)

• One-Hot Encoding was applied to categorical features, followed by Standard Scaling to normalize feature values.

• 5-fold cross-validation was used to improve reliability and reduce overfitting.

• Linear, RBF, and polynomial kernels were tested, and the polynomial kernel was selected as it achieved the highest overall performance across all metrics.

5-fold cross-validation results (present as table and mean ± std):

Kernel	Accuracy	F1 Score	Precision	Sensitivity (Recall)	Specificity
Linear	0.9340 ± 0.0099	0.9344 ± 0.0100	0.9358 ± 0.0102	0.8913 ± 0.0141	0.9689 ± 0.0057
RBF	0.9664 ± 0.0109	0.9668 ± 0.0110	0.9688 ± 0.0102	0.9278 ± 0.0536	0.9900 ± 0.0034
Poly	**0.9757 ± 0.0129**	**0.9753 ± 0.0136**	**0.9767 ± 0.0120**	**0.9190 ± 0.0581**	**0.9927 ± 0.0041**

Performance Evaluation:

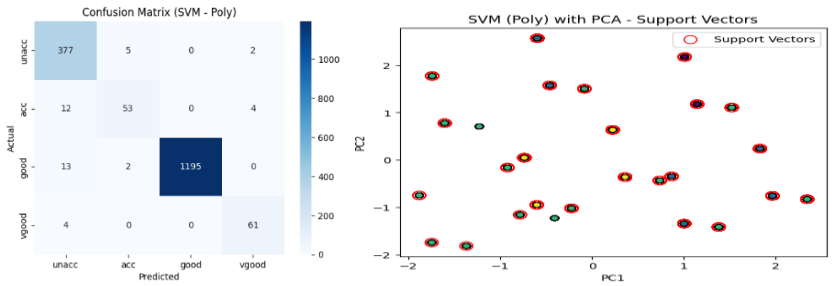
• The sensitivity (0.9190) indicates slightly lower performance in detecting minority classes compared to the RBF kernel.

• The high specificity (0.9927) shows that the model effectively identifies negative cases with very few false positives.

• The confusion matrix confirms that most predictions are correct, with minor misclassifications occurring mainly in smaller classes.

• The improvement from linear to non-linear kernels indicates that the dataset is not linearly separable.

• The relatively higher sensitivity of the RBF kernel suggests better performance on minority classes, while the polynomial kernel provides the best overall balance across all metrics.



Support vectors are the data points closest to the decision boundary and determine its position. The plot highlights these points (red circles). The large number of support vectors indicates a complex non-linear boundary due to the polynomial kernel and multi-class nature of the dataset. The stable cross-validation results suggest that the model generalizes well with limited overfitting.

4. Classification Algorithm 3 – Deep Learning (MLP) (10 points)

• The MLP model consists of an input layer, one hidden layer with 30 neurons using the ReLU function, which introduces non-linearity, improves computational efficiency, and mitigates the vanishing gradient problem, and an output layer corresponding to the target classes. This architecture is sufficient to capture non-linear relationships while reducing the risk of overfitting.
• The MLP model was trained using the Adam optimizer with a learning rate of 0.01, a mini-batch size of 200, and a maximum of 500 iterations (epochs).

MLP Network Architecture:

Layer	Number of Neurons	Activation Function	Description
Input Layer	Number of features (after encoding)	—	Receives scaled input features
Hidden Layer	30	ReLU	Learns non-linear relationships
Output Layer	4	SoftMax (implicit)	Produces class predictions

5-fold cross-validation results (present as table and mean ± std):

Model	Accuracy	F1 Score	Precision	Sensitivity (Recall)	Specificity
MLP	0.9936 ± 0.0034	0.9936 ± 0.0034	0.9939 ± 0.0032	0.9855 ± 0.0216	0.9977 ± 0.0010

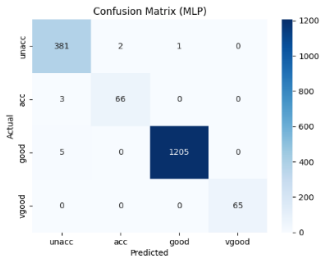
Performance Evaluation:

• The MLP model achieves very high accuracy and precision, indicating that it effectively learns complex nonlinear relationships in the dataset.

• The low standard deviation suggests that the model is stable and generalizes well with minimal overfitting.

• The very high specificity (0.9977) shows that the model produces very few false positives, demonstrating strong reliability in negative class predictions.

• The simple architecture (single hidden layer) was sufficient to achieve high performance, indicating that the dataset does not require a very deep network.



The near-perfect performance suggests the dataset is relatively simple after encoding.

5. Comparison and Conclusion (2 points)

Average Performance Metrics:

Model	Accuracy	F1 Score	Precision	Sensitivity (Recall)	Specificity
KNN (K=9)	0.9057	0.8981	0.9014	0.6751	0.9493
SVM (Poly)	0.9757	0.9753	0.9767	0.9190	0.9927
MLP	0.9936	0.9936	0.9939	0.9855	0.9977

- ***An accuracy comparison plot was generated to visualize the performance differences between models.***

Model	Advantages	Disadvantages
KNN	Simple and easy to implement; no training phase; intuitive; works well for small datasets	Lower accuracy; poor sensitivity for minority classes; slow prediction due to distance calculations; struggles with complex patterns
SVM (Poly)	High accuracy; effective for non-linear data using kernels; strong generalization; good balance between metrics	More complex, requires kernel selection; less interpretable than KNN
MLP	Highest accuracy; captures complex non-linear relationships; very high sensitivity and specificity; stable performance	Requires hyperparameter tuning; less interpretable; longer training time

Conclusion:

• Classification is an important task in machine learning with applications such as recommendation systems. In this project, MLP achieved the best performance, followed by SVM, while KNN had the lowest performance due to its limited ability to handle complex patterns and class imbalance, highlighting the importance of selecting appropriate models for complex data.

6. Video Demonstration (5 minutes max – 8 points)

• https://youtu.be/35EA_f0YlfI

Instructor Evaluation Checklist

Section	Max	Awarded	Criteria Met
Dataset Exploration and Preparation	10	[ ]	Dataset details, scaling, visualization, 5-fold setup
K-Nearest Neighbors (KNN)	10	[ ]	Scaling, K selection, cross-validation, metrics
Support Vector Machine (SVM)	10	[ ]	Scaling, kernel tests, support vectors, metrics
Deep Learning (MLP)	10	[ ]	Architecture, training params, cross-validation, metrics
Comparison and Conclusion	2	[ ]	Comparative table/plot + discussion and insights
Video Demonstration	8	[ ]	Clear and complete demo within 5 minutes
Total	**50**		